# SLIIT Codefest 2026

## AI Competition - Powered By IFS

**Project Title:** Ashen Era Archive Intelligent Document Assistant

**Team Name:** Code Wave

**Selected Challenge:** Sub-track 1C – Searching the Way a Human Does

**YouTube Video Link:** https://youtu.be/iG4RVT0yUmk

**Team Members:**


- K.V.G.M. Aradhya Jayawardane
- M.K.S.K. Kumara
- Pooja Anbalagan
- H.M. Sarasi Bimsara Rajarathna


---

### Problem Statement

- Enterprise document collections spread information across hundreds or thousands of pages and formats, so a single retrieval pass often answers only part of a question.
- Traditional RAG retrieves once and generates from that result, but complex questions need information connected across multiple documents and early results can reveal new entities or relationships that require further investigation.
- **Our approach models a human expert's search process:** Search → Read → Analyze → Identify Gaps → Search Again → Validate → Answer.

### Project Objective

- Develop an AI-powered document assistant that performs iterative, adaptive searches across a large corpus analyzing evidence, identifying missing information, and continuing to search until it can generate a complete, well-supported answer.

### Solution Overview

**Adaptive Multi-Step AI Document Assistant.**

- Unlike a traditional RAG system that retrieves once before answering, our system analyzes each search result, identifies evidence gaps, and uses what it learns to guide the next search.
- Workflow: A user question triggers an initial targeted search. Retrieved chunks are analyzed for sufficiency; if information is missing, the system identifies the gap, reformulates the query, and searches again.
- This continues: Search → Analyze → Identify Gaps → Reformulate → Search Again → Evaluate until evidence is sufficient, a stopping condition is met, or the iteration limit is reached.
- The consolidated evidence then generates the final answer, with source information preserved where possible for traceability.

**Key Innovation:** Rather than retrieving more chunks in one shot, each search's discoveries new entities, relationships, events, concepts actively shape the next search, letting the system progressively assemble information scattered across documents.

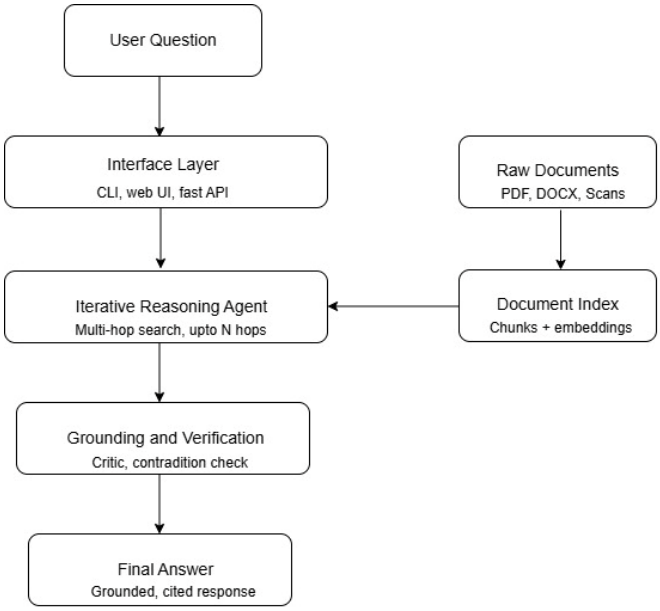
**Traditional RAG vs. Our Adaptive Search System**

**Traditional RAG: Question → Retrieve → Generate Answer**

**Our System:** Question → Search → Analyze → Identify Gaps → Reformulate → Search Again → Evaluate → Repeat → Generate Answer — an adaptive feedback loop between retrieval and reasoning that keeps searching while evidence is incomplete.

**System Architecture**

- The architecture supports a continuous cycle of searching, analyzing, identifying gaps, and following up until sufficient evidence is available or a stopping condition is reached.
- Stopping conditions: sufficient evidence collected; maximum iterations reached; no new relevant information found; results become repetitive; or a defined confidence threshold is met. Once stopped, evidence is consolidated into the final response.




**Architecture Components**

| | |
|---|---|
| **Document Processing Layer** | - This layer is responsible for loading and processing documents from the available document corpus. It prepares the source content for indexing and retrieval.<br>- Depending on the implemented system, the document collection may contain different formats and sources that must be converted into a searchable representation. |
| **Chunking and Indexing Layer** | - Documents are divided into smaller searchable units called chunks. Relevant metadata can be preserved alongside each chunk to improve traceability and retrieval.<br>- Possible metadata includes:<br>• Document name<br>• Source type<br>• Page number |

| | |
|---|---|
| | • Section<br>• Chunk ID<br>- The processed chunks are then stored or indexed to support efficient retrieval. |
| **Retrieval Layer** | - The Retrieval Layer searches the indexed document corpus using the current query.<br>- Depending on the implemented solution, retrieval may use:<br>• Semantic search<br>• Keyword search<br>• Hybrid search<br>• Vector similarity search<br>- The purpose of this layer is to retrieve the most relevant information for the current stage of the search process. |
| **Query Analysis and Planning Layer** | - This component analyzes the user's original question and determines the initial search direction.<br>- During later iterations, it can use information obtained from previous searches to determine what information is still missing and what should be searched for next.<br>- This component is central to the Sub-track 1C approach because it allows the search process to adapt based on newly discovered information. |
| **Reasoning and Evidence Analysis Layer** | - The retrieved information is analyzed to determine its relevance to the user's question.<br>- The system evaluates the evidence collected so far and identifies possible:<br>• Missing facts<br>• Missing entities<br>• Missing relationships<br>• Missing events<br>• Incomplete connections between information<br>- This analysis helps determine whether another search iteration is required. |
| **Sufficiency and Information Gap Evaluation** | - This component determines whether the currently collected evidence is sufficient to generate a complete answer.<br>- If important information is still missing, the system identifies the information gap and triggers the next search cycle.<br>- If sufficient evidence is available, the system proceeds to evidence aggregation and answer generation. |
| **Query Refinement and Next Search Planning** | - When the available evidence is incomplete, the system uses previously discovered information to formulate the next search action.<br>- The follow-up query may focus on:<br>• Newly discovered entities<br>• Related events<br>• Relationships between concepts<br>• Missing facts<br>• Additional supporting evidence<br>- This allows information discovered during one search to influence subsequent searches. |
| **Evidence Memory and Aggregation** | - Relevant information collected during multiple search iterations is retained and consolidated.<br>- This prevents the system from treating each search independently and supports the accumulation of evidence required to answer complex questions.<br>- Where implemented, this component can also help reduce repeated searches and preserve useful findings from previous retrieval steps. |
| **Answer Generation Layer** | - After the search process has finished, the collected evidence is consolidated and provided to the answer generation component.<br>- The final response is generated based on the accumulated evidence gathered across multiple search iterations. |

| | |
|---|---|
| | - Where supported by the implementation, source information can also be associated with the final answer to improve traceability. |

**Key Technical Decisions and Why We Made Them**

| Technical Decision | Why It Matters for Sub-track 1C |
|---|---|
| **1. Iterative Retrieval Instead of One-Shot Retrieval** | A single retrieval operation may provide only partial information for complex questions. We therefore adopted an iterative search approach in which the system can perform additional searches based on information discovered during previous retrieval steps. This better reflects the human-like information-seeking process required by Sub-track 1C. |
| **2. Query Reformulation** | Information retrieved during an initial search may reveal new entities, relationships, events, or concepts. The system can use these discoveries to guide follow-up searches, allowing later queries to be more targeted and context-aware. |
| **3. Evidence Sufficiency Evaluation** | Instead of automatically generating an answer after the first retrieval, the system evaluates whether the available evidence is sufficient. If important information is still missing, another search cycle can be initiated. |
| **4. Evidence Accumulation** | Complex questions may require information from multiple searches and documents. Relevant evidence collected across search iterations is retained and combined so that the final answer can be based on the accumulated information rather than a single retrieval result. |
| **5. Search Stopping Conditions** | Iterative searching can potentially continue unnecessarily or indefinitely. Defined stopping conditions help control the process, such as reaching a maximum number of iterations, detecting no meaningful new information, identifying repeated searches, or determining that sufficient evidence has been collected. |
| **6. Metadata Preservation** | Preserving metadata such as document name, source type, page number, section, or chunk identifier improves traceability. It helps identify where information originated and supports more credible, evidence-based answers. |

**Example of Adaptive Query Reformulation**

**Initial Question:** *Who was affected by Event X?*

The initial search may surface Person A and Organization B, but that alone may not be a complete answer. The system then runs targeted follow-up searches on Person A's involvement, Organization B's role, related events, and consequences mentioned elsewhere, progressively building a fuller understanding of the original question.

**Engineering Judgment**

Our central decision is to treat retrieval as an adaptive process rather than a single operation, where each search informs the next the core principle of Sub-track 1C: **Search → Learn → Identify Missing Information → Search Again → Evaluate → Answer.**

The final implementation should be judged on how well these decisions play out in practice, weighing answer completeness, retrieval quality, processing time, and number of search iterations.

**Results, Limitations, AI Usage and Team Contributions**

**What Works**:

- Multi-step search workflow
- Follow-up searches based on retrieved information
- Evidence accumulation across iterations
- Answering questions requiring multiple sources
- Search termination via defined stopping conditions
- Final answers grounded in retrieved evidence

**Limitations**

Possible areas: Ambiguous/complex questions, incorrect follow-up query generation, retrieval errors across interconnected documents, LLM reasoning limits, API rate limits, longer response times from multiple iterations, context limits, difficulty judging evidence sufficiency, redundant searches.

**Approaches Tried That Failed:** document each with Why We Tried It, What Happened, Why It Failed / Was Rejected, and What We Changed — based only on actual development evidence.

**AI Usage Disclosure:** AI tools were used as development assistants; the team made all final architectural, technical, implementation, testing, and validation decisions, reviewing, testing, modifying, or rejecting AI suggestions as needed. Full AI conversation logs are included in the Git repository per competition guidelines.

**Team Members and Contributions**

| Team Member | Role | Main Contributions |
|---|---|---|
| K.V.G.M. Aradhya Jayawardane | Data Engineering | • Ingest and process the data from the Ashen Era Archive (PDFs, DOCX, Markdown, and images), clean it, and load it into the vector database.<br>• Use OCR to read scanned pages and images, handle text chunking, and generate embeddings. |
| M.K.S.K. Kumara | AI/Backend Developer | • Write the core Python code for the RAG system or AI Agent.<br>• Integrate LLMs and embedding models (using Voyage AI and OpenRouter APIs), and implement the logic to fetch relevant data chunks and generate accurate answers when a user asks a question. |
| Pooja Anbalagan | Evaluation and Testing Lead | • Validate system accuracy and optimize performance.<br>• Test the system using the provided sample_questions.json (20 questions), identify bugs, and refine prompts to make the system faster and more accurate. |
| H.M. Sarasi Bimsara Rajarathna | Documentation, Git & Video Lead | • Manage competition compliance, documentation, and submissions.<br>• Maintain the Git repository discipline, save AI chat logs inside the ai_usage/ folder, prepare the 5-page PDF submission report, and record and upload the under-10-minute live demo video to YouTube as unlisted. |